

Análisis

En el desarrollo de software es muy común encontrarse con situaciones donde se deben crear objetos que no siempre son iguales, sino que pueden cambiar según lo que el usuario necesite. En este taller, el objeto principal es la hamburguesa, la cual hace parte de un sistema de pedidos de comida rápida. Aunque una hamburguesa parece algo simple, en realidad puede tener muchas combinaciones diferentes dependiendo de los gustos del cliente.

Toda hamburguesa debe tener ciertos elementos básicos para poder existir, como el tipo de pan y el tipo de carne. Estos elementos son obligatorios, ya que sin ellos no se podría considerar una hamburguesa completa. Además de estos elementos, también existen otros que no siempre están presentes, como el queso y los ingredientes adicionales. Estos últimos son opcionales porque cada cliente decide si los quiere o no.

El problema aparece cuando se intenta crear este tipo de objetos usando constructores normales. Si se hiciera de esta manera, habría que crear un constructor con muchos parámetros o varios constructores distintos para cubrir todas las combinaciones posibles. Esto haría que el código fuera difícil de entender, poco práctico y más propenso a errores, especialmente cuando se manejan muchos parámetros del mismo tipo.

Para evitar estos problemas, se utiliza el patrón de diseño Builder. Este patrón permite construir el objeto paso a paso, separando la lógica de construcción del objeto final. De esta forma, primero se definen los datos obligatorios y luego, si se desea, se agregan los datos opcionales. Esto hace que el código sea más claro y fácil de leer.

En este taller, la clase Hamburguesa se diseña como una clase inmutable, lo que significa que una vez que se crea una hamburguesa, ya no se puede modificar. Esto ayuda a que el objeto siempre se mantenga en un estado correcto y evita cambios inesperados. Para lograr esto, los atributos son privados, no existen métodos para modificarlos y el constructor es privado.

La creación de las hamburguesas se realiza únicamente a través de un Builder, el cual recibe los atributos obligatorios en su constructor y permite agregar los atributos opcionales mediante métodos. Finalmente, el método `build()` se encarga de crear la hamburguesa completa. Gracias a esta forma de trabajo, el código es más ordenado,

fácil de mantener y permite crear diferentes tipos de hamburguesas sin complicaciones.

En conclusión, el uso del patrón Builder es una solución adecuada para este problema, ya que permite manejar de manera sencilla la creación de hamburguesas con diferentes características.